The Journal of Open Source Software

auv: Vehicle abstraction and control system library for an autonomous underwater vehicle

Christopher Moran[1*¶]

1 Johns Hopkins University Applied Physics Laboratory, United States ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 31 October 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Abstract

auv is a vehicle abstraction library for autonomous underwater vehicles (AUVs) developed at the Johns Hopkins Applied Physics Laboratory (JHU/APL). It contains modules for critical vehicle subsystems including actuators, sensors, navigation, and flight control. This library leverages features including typestates for integrated compile-time checking of state machine transitions, deployment via a single cross-compiled binary, and a brokered message bus for automatic data logging to create an ergonomic, high-reliability base for building operator-out-of-the-loop autonomous systems. This library can be used as a dependency for developing higher-level mission code and/or autonomy for AUVs.

auv is generally decoupled from the initial vehicle hull it was developed and fielded on, and should be largely adaptable to a variety of autonomous underwater vehicles. AUVs running this software have been successfully deployed and recovered during trial missions.

Keywords

Statement of Need

Earth's surface is covered by over 70% water, resulting in a submerged area exceeding 350 million square kilometers.[^2] Relatively little of this area has been mapped (<30% as of 2025). Additionally, even ambitious projects like Seabed 2030 target mapping resolutions ranging from 100 x 100 m grid cells in the most shallow areas to 800 x 800 m grid cells in the deepest. While these measurements provide an important source of information, these resolutions are far below those needed for many important applications. Obtaining higher resolutions is, in large part, limited due to the acoustic physics involved with sensors mounted on surface vessels. To obtain human-scale, high-resolution maps of our world, the sensors themselves need to be positioned closer to the seabed. AUVs offer one of the only cost-effective methods for achieving this.

While many vehicles have been built by industry, academia, and government, AUVs remain limited and often fall far behind the capabilities of other platforms such as Uncrewed Aerial Systems (UAS), Surface Vessels (USVs), Ground Vehicles (UGVs), and even satellites. These can collectively be referred to as "UxVs". This is, in part, due to the lack of open systems that developers can use to iterate and improve upon existing systems. Virtually all AUVs commercially available today leverage proprietary, heavily locked-down software stacks that limit operators to a finite set of pre-programmed behaviors and actions such as waypoint or surveys. Even for advanced end users, vehicles only enable a "backseat driver" configuration, where a high-level autonomy can request certain parameters (such as a given heading, depth,

and speed) from the front-seat vehicle computer. The vehicle computer then acts as a black box responsible for reactive vehicle control.

Many users of these systems recognize these limitations and consider them non-ideal, but find there is no functional alternative. While open source flight control frameworks such as Ardupilot support a dizzying number of vehicle frames, including a variety of both multi-rotor and fixed-wing UAS, UGVs, USVs, and ROVs/vectored UUVs, there is no frame option for a torpedo-hulled autonomous underwater vehicle. As a result of this, many attempts at building a torpedo-hulled AUV capable of efficient lateral transit end up with non-traditional control schemas (Klemens, 2019).

Furthermore, past attempts at adapting other frames, such as fixed-wing UAS, to torpedo-hulled AUVs have proven difficult for a variety of reasons. AUVs operate in a fundamentally operator-out-of-the-loop paradigm. While submerged, virtually all radiofrequency (RF)-based communications signals are attenuated by the surrounding water within millimeters. Some level of on-mission communication is possible using acoustic communications (acomms). However, acomms links are very low-bandwidth (bits/second), high-latency (seconds), and often unreliable, making them sufficient to provide operators with a limited degree of situational awareness, not real-time operational control. Many other types of UxVs commonly expect reliable communication with their operator to be a standard part of their operational paradigm, and in some circumstances consider a lack thereof to be an indicator of faulted behavior. Similarly, many vehicles have an assumption of access to Global Positioning System (GPS) signals, which are also attenuated by water. Other UxV may also display characteristics that are not strongly shared by AUVs. AUVs are passively stable, run relatively slowly (1.5-2 m/s is typical), and have a relatively high inertial mass compared to their control inputs. These factors combined mean that they typically do not require high-rate control loops for normal operations.

There is a need for an AUV-focused control system that:

- Provides first-class support for unattended, operator-out-of-the-loop deployment
- Prioritizes error discovery and fault resolution at compile-time rather than runtime, when the autonomy developers and vehicle operators can observe, investigate, and fix issues
- Enables faster-than-real-time simulations that can be parallelized to explore a range of possible state spaces and failure modes
- Supports strong logging as an avenue for understanding vehicle behavior during post-mission analysis
- Provides modular interfaces that create a solid, extensible base for the construction of higher-level autonomy with the ability to transparently modify lower layers of the software and vehicle stack

auv is an experimental vehicle abstraction library for autonomous underwater vehicles designed to help address these needs.

Software Architecture

auv takes a "server-in-a-bottle" approach to autonomous systems development, where the software approach conceptualizes AUVs as servers embedded into pressure vessels that just so happen to have sensors and actuators attached as peripherals.

Rust Programming Language

The Rust programming language aims to be "a language empowering everyone to build reliable and efficient software," and supports these aims through a combination of language features that support compile-time assurances around static typing, mutability, memory management, and concurrency. These have helped to create a fast, memory- and thread-safe language with

DRAFT

excellent developer tooling. Robotics developers come from a variety of backgrounds and frequently do not have formal computer science training. The tooling-enforced correctness rather than implementation-enforced correctness allows developers who may otherwise have been unable to meaningfully engage with projects to contribute safely.

Idiomatic Rust code tends to push developers toward dealing with complexity upfront. For example, the pattern matching in Rust using the `match` statement is comprehensive, which means that developers must account for all possible outcomes of the match statement. If a new enumeration variant created during a refactor was not initially accounted for, this would likely raise a compile-time error, not undefined or non-deterministic behavior at runtime.

This applies to memory safety as well, where memory safety-related errors leading to undefined behavior can ultimately result in vehicle loss. Rust's emphasis on correctness-by-design limits the potential for undefined behaviors and allows a greater range of contributors, who otherwise may have been unable to meaningfully engage in hands-on development, to contribute with less risk. Modern embedded Linux single-board computers more often lean towards many slower cores than fewer fast ones. The thread safety enforced by the Rust compiler enables developers to write fast, correct, multi-threaded code that can more efficiently use the limited resources available on these systems.

Additionally, the Rust ecosystem's tooling has played a significant role. For large codebases in other languages, it's not uncommon for developer onboarding to take several days, as developers work to manage an extensive list of dependencies installed across their system or complex hybrid build systems. During several internal review sessions for `auv`, new developers were consistently able to clone the Git repository, build and modify the `minimal` example mission, cross-compile the mission for a development vehicle flight control target architecture, push the mission to the vehicle, trigger the mission, and post-process the logs in under <1 hour. The ability to both onboard and quickly engage with a hardware-rich development cycle greatly decreases risk across the project lifecycle.

Typestates

Many autonomous systems rely heavily on state machines. State machines are systems that can be in exactly one of a finite number of states at any given time. As systems evolve and gain complexity over time, states can be more difficult to manage. The typestate pattern is a method that uses the Rust type system to enforce constraints on valid state machine transitions at compile-time rather than at run-time. This allows related sections of code to be semantically connected while managing possible state variations. The use of typestates in the context of a robotic system also enables the correct initialization of certain unique resources.

For example, when transitioning from `Auv<Idle>` to `Auv<Active>`, which allows access to certain methods that developers should only use while the vehicle is fully initialized and underway(Duarte & Ravara, 2021). An example of this is the `Thruster`, which undergoes hardware initialization of the Electronic Speed Controller (ESC) when the vehicle transitions between the `Idle` and `Active` states; even if the `Auv::thruster()` method was available, it should not be used by developers during `Idle`, as the incorrect start-up sequence can lead to unexpected thruster behavior upon subsequent inputs. The use of typestates to formally encode and enforce these transitions helps provide a base upon which developers can more safely and deliberately build complex mission-level behaviors.

Interprocess Communication (IPC)

Interprocess communication (IPC) is one of the fundamental problems in robotics software development, where the complexity of these systems naturally drives the modularization of code that handles discrete subsystems or activities, but still needs a way to interact with one another.

The `meadow` library is used for this function. It primarily uses a request-response architecture,

where components request data from a central broker as needed. This increases the level of both agency and responsibility that a component has over its data, and differs in approach from the event-driven publish-subscribe architectures commonly found in robotics frameworks. `meadow` enables the transport of `serde`-compatible data types into the `postcard` wire format. Since many of the libraries that `auv` uses already support `serde` (such as the Math Interoperability Standard Types `mint` library), `meadow` natively supports these types without needing to resort to parsing message format definitions such as `.idl` or `.proto`. This does, however, come at the trade-off of making inter-language operation more difficult. One common concern regarding data buses is the latency involved in accessing stored information. Stochastic benchmarks (via `criterion`) show `meadow` is capable of performing round-trip publish-request-receive packet operations on `localhost` using a `Node` and `Host` with average latencies in the low 10's of microseconds, or <10 microseconds for direct data inserts/retrievals using `Host`-direct methods.

Within `auv`, the integrated sensor with the highest available data rate is the Bosch BNO055, which is described as a "smart sensor combining accelerometer, gyroscope, magnetometer and orientation software" and is the primary method of obtaining attitude information. The BNO055 operates at a maximum data rate of 100 Hz. When compared to other flight platforms, such as multi-rotor UAVs, AUV flight systems require an extremely low control rate for stable flight. This was demonstrated by the Monterey Bay Aquarium Research Institute's Tethys Long-Range Autonomous Underwater Vehicle (MBARI Tethys LRAUV). The Tethys control loop nominally ran at a rate of 2.5 Hz (Hobson et al., 2012). Even for vehicles running a control loop an order of magnitude faster, the demonstrated latency is functionally below the noise floor for all other operations and has not led to a noticeable effect on the stability of the `auv` flight control algorithm, even during faster-than-real-time simulations. This has demonstrated the viability of this approach to a message bus for soft real-time AUV control loops.

Logging

The `auv` library supports two types of logging:

1) The `meadow` bus directly logs all `postcard`-serialized data that it receives based on the message timestamp. This information tends to relate to vehicle-level inputs and outputs, such as sensor information or the oscillation of a vehicle around a setpoint. The `auv` library package supports post-processing this information using the `tools/log-processing` toolkit, which includes traits and methods that can be used to support processing of arbitrary `meadow`-compatible data types.

2) The `tracing` library, which is "a framework for instrumenting Rust programs to collect structured, event-based diagnostic information." This information tends to relate to software implementation of the system itself, including non-critical errors or other debug information for software developers.

Software Bill of Materials and Software Supply Chain

In addition to its function as the standard Rust build system and package manager, `cargo` also enables adding extensions that support building Software Bills of Materials. This has become an increasing area of focus, and the Rust language's in-directory builds, semantic-versioned dependencies, and default statically-compiled binaries provide a strong base for being able to comprehensively define the inputs to mission-critical software.

Additionally, the `cargo-cyclonedx` tool enables developers to quickly and correctly build a CycloneDX-formatted Software Bill of Materials document that includes every dependency down to the specific compiler toolchain and statically-compiled `musl` libc implementation. This toolset helps to meet a need in today's modern software environment for deployable systems, helps to support meeting recommendations made by the Cybersecurity and Infrastructure Security Agency (CISA) around vulnerability management (Muirí, 2019).

This has positive side effects for the development process as well. During the development of auv, upstream packages twice broke semantic versioning that led to project-level build failures. While these were in both cases accidental and quickly fixed, `cargo`'s ability to easily specify specific package version numbers allowed the development to resume working within minutes of the disruption.

Experimental Results

The auv library has been used on several vehicles, ranging from 6 to 8.5 inches in diameter, but all in the small-displacement vehicle category. Much of the testing performed with these vehicles up to this point has been developmentally-focused, with a particular focus on proving the operation of the core attitude control loop in an `Active` Auv. An sample graph from an open water mission, comprising:

1. Released from the surface by an operator
2. Yaw setpoint for the current heading (straight-ahead)
3. 0.5 m depth setpoint for 30 seconds
4. 10.0 m depth setpoint for 2 minutes
5. 0.0 m depth setpoint (surfacing action) for 30 seconds

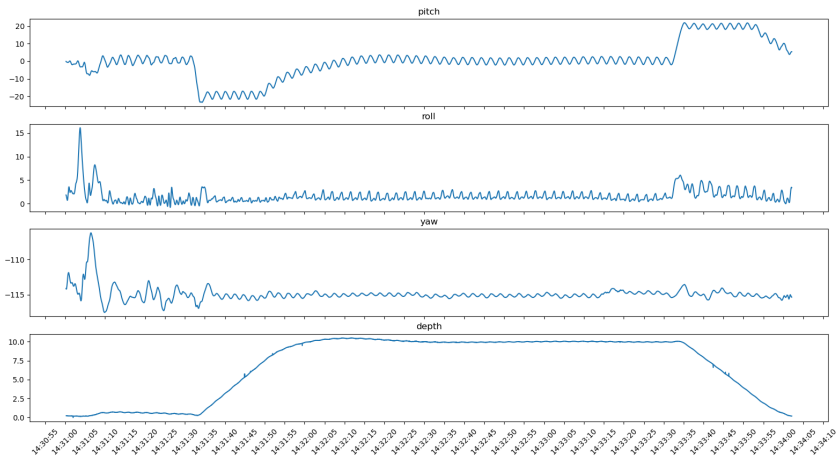

Figure 1: Subset of mission logs; 3-min, straight-line, 0.5m-10.m-0.0m depth profiles

This profile demonstrates successful control of the vehicle across roll, pitch, yaw, and depth (albeit with a slightly underdamped pitch controller contributing to oscillatory pitch). While the total duration on this mission was slightly over three minutes, successful missions have been completed that exceed 10 minutes in duration, covering over 1 km of submerged travel, and demonstrating successful heading changes while underway. While these missions are still significantly shorter than would be necessary for large-scale ocean mapping or similar profiles, it does demonstrate proof-of-concept as a viable control system for an autonomous underwater vehicle.

Acknowledgements

Thank you to the JHU/APL Janney Grants program for providing the funding to publish this work. This would not have been possible otherwise.

Citations

- Duarte and Ravara (2021)
- Klemens (2019)
- Muiri (2019)
- Hobson (2012)

Duarte, J., & Ravara, A. (2021). Retrofitting typestates into rust. *25th Brazilian Symposium on Programming Languages*, 83–91. https://doi.org/10.1145/3475061.3475082

Hobson, B. W., Bellingham, J. G., Kieft, B., McEwen, R., Godin, M., & Zhang, Y. (2012). Tethys-class long range AUVs-extending the endurance of propeller-driven cruising AUVs from days to weeks. *2012 IEEE/OES Autonomous Underwater Vehicles (AUV)*, 1–8.

Klemens, K. (2019). 6" diameter AUV hull for r&d. In *Discuss forum post*. Blue Robotics Discuss. https://discuss.bluerobotics.com/t/6-diameter-auv-hull-for-r-d/5300

Muirí, É. Ó. (2019). Framing software component transparency: Establishing a common software bill of material (sbom). *NTIA, Nov*, *12*.

DRAFT